

Lab 10 ชาคริต รุ่งเรืองงาม 673380399-7 Sec 3

ตอบคำถาม

1. นักศึกษาอธิบายได้ว่า Reactive Programming คืออะไร และแตกต่างจาก Blocking I/O อย่างไร

Reactive Programming คืออะไร และแตกต่างจาก Blocking I/O อย่างไร?

Reactive Programming คือ รูปแบบการเขียนโปรแกรมเชิงตอบสนองต่อสายข้อมูล (Data Stream) แบบ Asynchronous และ Non-blocking โดยโปรแกรมจะไม่จอดรอดำเนินการจนเสร็จ แต่จะลงทะเบียนการทำงานไว้ (Subscribe) เมื่อมีข้อมูลหรือเหตุการณ์เกิดขึ้น (Event) ระบบจึงจะเรียกประมวลผลต่อทันที

ข้อแตกต่างระหว่าง Reactive Programming และ Blocking I/O:

- Blocking I/O (เช่น Spring MVC เดิม): ทำงานแบบ 1 Thread ต่อ 1 Request หากการทำงานต้องรอดึงข้อมูลจาก Database หรือ External API ตัว Thread จะถูกแขวนรอ (Block) ทำให้สิ้นเปลืองทรัพยากรเมื่อมีผู้ใช้งานพร้อมกันจำนวนมาก
- Non-blocking I/O (เช่น Spring WebFlux / Event Loop): ใช้ Thread จำนวนน้อยในการรับ Request งานที่มีระยะเวลารอนานจะถูกส่งไปทำเบื้องหลังโดยไม่แขวน Thread ไว้ ทำให้ Thread สามารถไปรับ Request อื่นต่อได้ทันที จึงรองรับการทำงานพร้อมกันจำนวนมาก (High Concurrency) และใช้อุปกรณ์ได้อย่างคุ้มค่า

2. นักศึกษาเข้าใจ **Mono<T>** และ **Flux<T>** และรู้ว่าควรใช้แบบไหนในสถานการณ์ใด

ทั้งสองคลาสเป็น Publisher (ผู้ส่งสตรีมข้อมูล) ตามมาตรฐาน Reactive Streams ใน Project Reactor:

- Mono

ความหมาย : สายข้อมูลที่ส่งออกผลลัพธ์ได้ตั้งแต่ 0 ถึง 1 รายการ (Zero or One Element)

สถานการณ์ที่ใช้ : ใช้สำหรับงานที่คืนค่าเพียงรายการเดียว เช่น การค้นหาสินค้าจาก ID (getProductById), การบันทึกข้อมูล (saveProduct) หรือการคืนสถานะการลบข้อมูล (Mono)

- Flux

ความหมาย : สายข้อมูลที่ส่งออกผลลัพธ์ได้ตั้งแต่ 0 ถึง N รายการ (Zero to N Elements / Sequence)

สถานการณ์ที่ใช้ : ใช้สำหรับงานที่คืนค่าเป็นชุดข้อมูลหรือหลายรายการ เช่น การดึงรายการสินค้าทั้งหมด (getAllProducts) หรือการกรองสินค้าตามหมวดหมู่ (getByCategory)

3. นักศึกษาสร้าง REST API ด้วย **Spring WebFlux** และ **@RestController** ได้

ใน Spring WebFlux การสร้าง REST API จะใช้ Annotation ตัวเดียวกับ Spring MVC เช่น **@RestController** และ **@RequestMapping("/products")** แต่จุดต่างคือการคืนค่าใน Controller Method จะต้องส่งกลับเป็น Reactive Types (Mono หรือ Flux) เพื่อรองรับการทำงานแบบ Non-blocking เช่น:

- `@GetMapping("/{id}") public Mono getById(@PathVariable String id) { return service.getById(id); }`
- `@GetMapping public Flux getAll() { return service.getAll(); }`

4. นักศึกษาใช้ **WebClient** เรียก HTTP API แบบ non-blocking ได้

WebClient เป็น HTTP Client ของ Spring WebFlux ที่มาแทนที่ RestTemplate ทำหน้าที่รับ-ส่ง HTTP Request แบบ Non-blocking และ Asynchronous ผ่าน Method Chain เช่น:

```
public Mono getProductById(String id) { return client.get().uri("/products/{id}", id).retrieve().bodyToMono(Product.class); }
```

ขั้นตอนการทำงาน:

1. `client.get()` — กำหนด HTTP Method เป็น GET
2. `.uri(...)` — กำหนด URL Endpoint
3. `.retrieve()` — เริ่มกระบวนการส่ง Request และรับ Response
4. `.bodyToMono(...)` — แปลงข้อมูลใน Body ให้เป็นออบเจกต์ Mono
5. การใช้ Operators ที่สำคัญใน Reactive Streams

- `.map()`: ใช้แปลงข้อมูลแบบ Synchronous (1 ต่อ 1) เช่น แปลงออบเจกต์ Product ให้เป็นราคาหลังส่วนลด Double (`Product::getDiscountedPrice`)
- `.flatMap()`: ใช้แปลงข้อมูลแบบ Asynchronous ที่คืนค่ากลับมาเป็น Publisher (Mono หรือ Flux) อื่น
- `.filter()`: ใช้กรองข้อมูลสตรีมเฉพาะรายการที่ผ่านเงื่อนไข boolean เช่น `.filter(p -> p.getCategory().equalsIgnoreCase(category))`
- `.subscribe()`: สั่งให้ Reactive Pipeline เริ่มประมวลผลข้อมูล (โดยทั่วไปใน Spring WebFlux ตัว Framework จะทำการ subscribe ให้โดยอัตโนมัติเมื่อส่งข้อมูลกลับไปยัง Client)

5. นักศึกษาใช้ Operators สำคัญ (`map`, `flatMap`, `filter`, `subscribe`) ได้ถูกต้อง

`.map()` (Data Transformation):

- **ลักษณะการทำงาน:** ใช้สำหรับการแปลงข้อมูลรูปแบบ Synchronous แบบ 1 ต่อ 1 (แปลงค่าจาก T เป็น V)
- **การใช้งานในแลบ:** ใช้ในเมธอด `getDiscountedPrice()` เพื่อแปลงออบเจกต์ Product ให้เป็นค่าราคาหลังหักส่วนลดประเภท Double

Java

```
public Mono<Double> getDiscountedPrice(String id) {

    return getByid(id)

        .map(Product::getDiscountedPrice);

}
```

`.flatMap()` (Asynchronous Transformation):

- **ลักษณะการทำงาน:** ใช้สำหรับการแปลงข้อมูลเมื่อผลลัพธ์ของการแปลงคืนค่ากลับมาเป็น Publisher (Mono หรือ Flux) อีกตัวหนึ่ง โดยทำการคลี่/ยุบ (Flatten) สายข้อมูลซ้อนให้อยู่ในสตรีมเดียวกัน
- **การใช้งานในแลบ:** ใช้ในกรณีที่ต้องการต่อ Pipeline Asynchronous เช่น การดึงข้อมูล Product แล้วนำ ID ไปค้นหาข้อมูลอื่นต่อในรูปแบบ Reactive

.filter() (Data Filtering):

- **ลักษณะการทำงาน:** ใช้สำหรับการคัดกรองข้อมูลในสตรีม ให้ผ่านเฉพาะรายการที่ตรงตามเงื่อนไขที่กำหนด (คืนค่าเป็น boolean)
- **การใช้งานในแลบ:** ใช้ในเมธอด `findByCategory()` เพื่อดึงเฉพาะสินค้าที่มีหมวดหมู่ตรงกับที่ระบุ

Java

```
public Flux<Product> findByCategory(String category) {  
  
    return findAll()  
  
        .filter(p -> p.getCategory() != null && p.getCategory().equalsIgnoreCase(category));  
  
}
```

.subscribe() (Stream Execution):

- **ลักษณะการทำงาน:** สตรีมข้อมูลใน Reactive Programming จะเป็นแบบ Lazy (ไม่ทำงานจนกว่าจะมีผู้ติดตาม) คำสั่ง `.subscribe()` จึงเป็นตัวสั่งให้ Reactive Pipeline เริ่มไหลและประมวลผลข้อมูลจริง
- **การใช้งานในแลบ:** ในฝั่ง Controller ของ Spring WebFlux ตัว Framework จะทำการกด `.subscribe()` ให้โดยอัตโนมัติเมื่อส่ง Response กลับไปยัง Web Client / Browser ส่วนในการใช้งาน Web Client หรือการทดสอบ จะมีการกด `.subscribe()` หรือคืนสตรีมเพื่อให้ผู้เรียกนำไปประมวลผลต่อ

GET | /products/{id} | Mono<Product> |  ทำแล้ว (ตัวอย่าง)

Home

Workspaces

Explore

Search Postman

Sign In

Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History

New Import

GET http://localhost:8080/products/1

http://localhost:8080/products/1

Save

</>

GET

http://localhost:8080/products/1

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body

Cookies

Headers (2)

Test Results

200 OK 15 ms 237 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

Copy

```
1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (673388399-7)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 50,
7   "price": 39900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 35910.0
10 }
```

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

Console

Not connected to a Postman account

GET | /products | Flux<Product> |  TODO

Home

Workspaces

Explore

Search Postman

Sign In

Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History

New Import

GET http://localhost:8080/products

http://localhost:8080/products

Save

</>

GET

http://localhost:8080/products

Send

Params

Authorization

Headers (6)

Body

Pre-request Script

Tests

Settings

Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body

Cookies

Headers (2)

Test Results

Status: 200 OK Time: 7 ms Size: 508 B

Save Response

Pretty

Raw

Preview

Visualize

JSON

Copy

```
2 {
3   "id": "1",
4   "name": "iPhone 15 Pro (673388399-7)",
5   "category": "Electronics",
6   "brand": "Apple",
7   "stock": 50,
8   "price": 39900.0,
9   "discountType": "MEMBER",
10  "discountedPrice": 35910.0
11 },
12 {
13   "id": "2",
14   "name": "MacBook Air M3",
15   "category": "Electronics",
16   "brand": "Apple",
17   "stock": 20,
18   "price": 49900.0,
19   "discountType": "NONE",
20   "discountedPrice": 49900.0
21 },
22 {
23   "id": "3",
24   "name": "Samsung Galaxy S24",
25   "category": "Electronics",
26   "brand": "Samsung",
27   "stock": 30,
28   "price": 29900.0,
29   "discountType": "SEASONAL",
30   "discountedPrice": 23920.0
31 }
```

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

Console

Not connected to a Postman account

POST

/products

Mono<Product>

X TODO

Home Workspaces Explore Search Postman

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- POST http://localhost:8080/products
- GET http://localhost:8080/products
- POST http://localhost:8080/products
- POST http://localhost:8080/products
- GET http://localhost:8080/products
- GET http://localhost:8080/products/1

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

POST http://localhost:8080/products

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "id": "4",
3   "name": "RTX5899 T2",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 88,
7   "price": 39999.8,
8   "discountType": "MEMBER",
9   "discountPrice": 39919.8
10 }
```

Body Cookies Headers (2) Test Results

Status: 200 OK Time: 24 ms Size: 220 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "id": "4",
3   "name": "RTX5899 T2",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 88,
7   "price": 39999.8,
8   "discountType": "MEMBER",
9   "discountPrice": 39919.8
10 }
```

Console Not connected to a Postman account

DELETE

/products/{id}

Mono<Void>

X TODO

Home Workspaces Explore Search Postman

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import

Today

- DEL http://localhost:8080/products/4
- DEL http://localhost:8080/products/4
- POST http://localhost:8080/products
- GET http://localhost:8080/products
- POST http://localhost:8080/products
- GET http://localhost:8080/products
- GET http://localhost:8080/products/1

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

DEL http://localhost:8080/products/4

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1
```

Body Cookies Headers (1) Test Results

Status: 200 OK Time: 4 ms Size: 38 B Save Response

Pretty Raw Preview Visualize Text

```
1
```

Console Not connected to a Postman account

GET

/products/category/{cat}

Flux<Product>

X TODO

Home Workspaces Explore Search Postman

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History

- GET http://localhost:8080/products/category/Electronics
- DEL http://localhost:8080/products/4
- DEL http://localhost:8080/products/4
- POST http://localhost:8080/products
- GET http://localhost:8080/products
- POST http://localhost:8080/products
- POST http://localhost:8080/products
- GET http://localhost:8080/products
- GET http://localhost:8080/products/1

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

GET http://localhost:8080/products/category/Electronics

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Body

```
1 {
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (67336399-7)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 50,
8     "price": 3999.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 3599.0
11  },
12  {
13    "id": "2",
14    "name": "MacBook Air M3",
15    "category": "Electronics",
16    "brand": "Apple",
17    "stock": 20,
18    "price": 4999.0,
19    "discountType": "NONE",
20    "discountedPrice": 4999.0
21  },
22  {
23    "id": "3",
24    "name": "Samsung Galaxy S24",
25    "category": "Electronics",
26    "brand": "Samsung",
27    "stock": 30,
28    "price": 2999.0,
29    "discountType": "SEASONAL",
30    "discountedPrice": 2799.0
31  }
32 }
```

Status: 200 OK Time: 7 ms Size: 558 B Save Response

GET

/products/{id}/price

Mono<Double>

X TODO

Home Workspaces Explore Search Postman

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History

- GET http://localhost:8080/products/1/price
- GET http://localhost:8080/products/category/Electronics
- DEL http://localhost:8080/products/4
- DEL http://localhost:8080/products/4
- POST http://localhost:8080/products
- GET http://localhost:8080/products
- POST http://localhost:8080/products
- POST http://localhost:8080/products
- GET http://localhost:8080/products
- GET http://localhost:8080/products/1

Create collections in Postman

Use collections to save your requests and share them with others.

Create a Collection

GET http://localhost:8080/products/1/price

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Body

```
1 3599.0
```

Status: 200 OK Time: 6 ms Size: 77 B Save Response

1. Reactive Programming vs Blocking I/O — ต่างกันอย่างไร

หัวข้อ เปรียบเทียบ	Blocking I/O (เช่น Spring MVC)	Reactive Programming / Non-blocking (เช่น Spring WebFlux)
รูปแบบการ ทำงาน	Thread-per-Request Model 1 Request จะใช้ 1 Thread ตั้งแต่เริ่มจนจบ	Event-Driven / Event Loop Model ใช้ Thread จำนวนน้อย (เท่ากับจำนวน CPU Cores) คอยจัดการ Events
การจัดการเมื่อ เจอ I/O	Blocking (แขวนรอ) เมื่อต้องรอ Database หรือ External API ตัว Thread จะหยุดนิ่งรอจนกว่าจะได้ Response คืนมา	Non-blocking (ไม่แขวนรอ) เมื่อส่ง Request ไปยัง I/O แล้ว Thread จะปล่อยให้ I/O ทำงานเบื้องหลัง แล้วไปรับ Request อีกต่อทันที
การใช้ทรัพยากร (Resources)	ใช้ Memory และ Thread สูง หากมี Request เข้ามาเยอะจน Thread Pool เต็ม ระบบจะช้าลงหรือค้าง (Thread Exhaustion)	ประหยัด Memory และใช้ Thread ค่อนข้างน้อย รองรับผู้ใช้งานจำนวนมาก พร้อมกันได้ดี (High Concurrency)

หัวข้อ เปรียบเทียบ	Blocking I/O (เช่น Spring MVC)	Reactive Programming / Non-blocking (เช่น Spring WebFlux)
เหมาะกับงาน ประเภทใด	งาน CRUD ทั่วไปที่มีผู้ใช้ไม่เยอะมาก หรือตรรกะซับซ้อนที่เน้นเขียนง่าย	ระบบที่รองรับ Traffic สูง, ระบบ Streaming, Microservices ที่ต้อง ต่อเรียก API หลายตัวพร้อมกัน

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ทั้ง `Mono<T>` และ `Flux<T>` เป็นการนำแนวคิด **Reactive Streams (Publisher)** มาใช้งานผ่านไลบรารี Project Reactor ใน Java โดยมีข้อแตกต่างและการใช้งานดังนี้:

2.1 Mono<T>

- **คืออะไร:** สตริมข้อมูลที่ส่งออกผลลัพธ์ได้ 0 ถึง 1 รายการ (Zero or One Element)
- **ใช้กรณีไหน:**
 - การค้นหาข้อมูลตาม Primary Key/ID ที่รู้แน่นอนว่าเจอสูงสุด 1 ชิ้น เช่น `findById(id)`
 - การบันทึกข้อมูลใหม่แล้วส่งคืนออบเจกต์ที่สร้าง เช่น `save(product)`
 - เมธอดที่ไม่ต้องการส่งค่ากลับ แต่ต้องการแจ้งสถานะว่าเสร็จสมบูรณ์ (`Mono<Void>`)
- **ทำไมถึงต้องใช้:** เพื่อสื่อสารความตั้งใจของโค้ด (Intention-revealing API) ชัดเจนว่า Endpoint หรือ Business Logic นี้จะคืนค่าออบเจกต์เดียว ช่วยให้ Framework และระบบจัดการ Memory ได้อย่างเหมาะสมสำหรับ Data Item เดียว

2.2 Flux<T>

- **คืออะไร:** สตริมข้อมูลที่ส่งออกผลลัพธ์ได้ตั้งแต่ 0 ถึง N รายการ (Zero to N Elements / Sequence)
- **ใช้กรณีไหน:**
 - การดึงรายการข้อมูลทั้งหมด เช่น `findAll()`

- การค้นหาหรือกรองข้อมูลที่อาจเจอหลายรายการ เช่น `findByCategory(category)`
- ระบบที่ต้องดึงข้อมูลแบบ Real-time Stream (Server-Sent Events: SSE)
- **ทำไมถึงต้องใช้:** เพื่อรองรับการส่งข้อมูลแบบลำดับ (Stream) แบบ Non-blocking ซึ่งช่วยให้ระบบสามารถทยอยส่งข้อมูลที่ละชิ้นไปยัง Client ได้ทันทีโดยไม่ต้องรอโหลดข้อมูลทั้งหมดใส่ List ใน Memory ก่อน

3. WebClient — อธิบาย Method Chain

WebClient เป็น HTTP Client แบบ Non-blocking และ Asynchronous ใน Spring WebFlux ที่สร้างขึ้นมาจาก RestTemplate โดยมีรูปแบบการทำงานผ่าน Method Chain ดังนี้:

Java

```
public Mono<Product> getProductById(String id) {

    return webClient.get()           // Step 1

        .uri("/products/{id}", id)   // Step 2

        .retrieve()                  // Step 3

        .bodyToMono(Product.class);  // Step 4

}
```

- **Step 1: .get() (หรือ .post(), .put(), .delete())**
 - **หน้าที่:** กำหนด HTTP Method ที่ต้องการส่งไปยังเซิร์ฟเวอร์ปลายทาง
- **Step 2: .uri("/products/{id}", id)**
 - **หน้าที่:** กำหนด Path และ Parameters ของ URL ปลายทาง โดยระบบจะนำค่าใน Argument (id) ไปแทนที่ตัวแปรใน URI ให้อัตโนมัติ
- **Step 3: .retrieve()**

-
- // ดึงสินค้าทั้งหมด คืนค่าเป็น Flux (0..N)
- public Flux<Product> findAll() {
- return Flux.fromIterable(productMap.values());
- }
-
- // ดึงสินค้าตาม ID คืนค่าเป็น Mono (0..1)
- public Mono<Product> findById(String id) {
- return Mono.justOrEmpty(productMap.get(id));
- }
-
- // บันทึกหรืออัปเดตสินค้า
- public Mono<Product> save(Product product) {
- productMap.put(product.getId(), product);
- return Mono.just(product);
- }
-
- // ลบสินค้าตาม ID คืนค่าเป็น Mono<Void> เพื่อบอกสถานะการทำงานสำเร็จ
- public Mono<Void> deleteById(String id) {
- productMap.remove(id);
- return Mono.empty();
- }
- }

• **4.2 ProductService.java (Business Logic Layer)**

- Java
- package com.example.lab10.service;
-
- import com.example.lab10.model.Product;
- import com.example.lab10.repository.ProductRepository;
- import org.springframework.stereotype.Service;
- import reactor.core.publisher.Flux;
- import reactor.core.publisher.Mono;
-
- import java.util.UUID;
-
- @Service
- public class ProductService {
-
- private final ProductRepository repository;
-
- public ProductService(ProductRepository repository) {
- this.repository = repository;
- }
-
- // ดึงรายการสินค้าทั้งหมด
- public Flux<Product> getAllProducts() {
- return repository.findAll();
- }

-
- // ค้นหาสินค้าตาม ID หากไม่พบจะคืนค่า Empty Mono
- public Mono<Product> getProductById(String id) {
- return repository.findById(id);
- }
-
- // กรองสินค้าตามหมวดหมู่ โดยใช้ Operator .filter()
- public Flux<Product> getProductsByCategory(String category) {
- return repository.findAll()
- .filter(p -> p.getCategory() != null &&
- p.getCategory().equalsIgnoreCase(category));
- }
-
- // คำนวณราคาหลังหักส่วนลด โดยใช้ Operator .map()
- public Mono<Double> getDiscountedPrice(String id) {
- return repository.findById(id)
- .map(Product::getDiscountedPrice); // แปลงออบเจกต์ Product ให้
- เป็นราคาหลังส่วนลด (Double)
- }
-
- // สร้างสินค้าใหม่ หากไม่ได้ระบุ ID มา ระบบจะสุ่ม UUID ให้อัตโนมัติ
- public Mono<Product> createProduct(Product product) {
- if (product.getId() == null || product.getId().isEmpty()) {
- product.setId(UUID.randomUUID().toString());
- }
- return repository.save(product);
- }
-
- // ลบสินค้าตาม ID
- public Mono<Void> deleteProduct(String id) {
- return repository.deleteById(id);
- }
- }

• **4.3 ProductController.java (REST Controller Layer)**

- Java
- package com.example.lab10.controller;
-
- import com.example.lab10.model.Product;
- import com.example.lab10.service.ProductService;
- import org.springframework.http.HttpStatus;
- import org.springframework.web.bind.annotation.*;
- import reactor.core.publisher.Flux;
- import reactor.core.publisher.Mono;
-
- @RestController
- @RequestMapping("/products")
- public class ProductController {
-

```

• private final ProductService service;
•
• public ProductController(ProductService service) {
•     this.service = service;
• }
•
• // GET /products : ดึงสินค้าทั้งหมดในรูปแบบ Non-blocking Flux
• @GetMapping
• public Flux<Product> getAllProducts() {
•     return service.getAllProducts();
• }
•
• // GET /products/{id} : ดึงสินค้าตาม ID ในรูปแบบ Mono
• @GetMapping("/{id}")
• public Mono<Product> getProductById(@PathVariable String id) {
•     return service.getProductById(id);
• }
•
• // GET /products/category/{category} : ดึงสินค้าที่รายการก็ได้ที่ตรงกับหมวดหมู่
• @GetMapping("/category/{category}")
• public Flux<Product> getProductsByCategory(@PathVariable String
category) {
•     return service.getProductsByCategory(category);
• }
•
• // GET /products/{id}/price : ดึงราคาหลังหักส่วนลดของสินค้าตาม ID
• @GetMapping("/{id}/price")
• public Mono<Double> getDiscountedPrice(@PathVariable String id) {
•     return service.getDiscountedPrice(id);
• }
•
• // POST /products : รับ JSON แล้วสร้างสินค้าใหม่ พร้อมคืนสถานะ HTTP 201 Created
• @PostMapping
• @ResponseStatus(HttpStatus.CREATED)
• public Mono<Product> createProduct(@RequestBody Product product) {
•     return service.createProduct(product);
• }
•
• // DELETE /products/{id} : ลบสินค้าตาม ID พร้อมคืนสถานะ HTTP 204 No Content
• @DeleteMapping("/{id}")
• @ResponseStatus(HttpStatus.NO_CONTENT)
• public Mono<Void> deleteProduct(@PathVariable String id) {
•     return service.deleteProduct(id);
• }
• }

```